



Technical Design Document

A bookkeeping assistant that helps a bookkeeper extract receipt details, classify expenses and compare bank transactions, while keeping a human in the loop for uncertain decisions.

TEAM	MJJ Kakis
TEAM CODE	XWMUE176
GITHUB REPOSITORY	github.com/yipkaio/Ledgerly
PUBLISHED	25 September 2026

1. Project overview

Problem: In a small business, receipts and bank transactions often arrive separately. A bookkeeper must read each receipt, choose a consistent expense category and later compare it with a bank debit. Repeated entry and inconsistent categorisation can create avoidable review work.

Approach: Ledgerly extracts receipt details, suggests categories and compares accepted receipts with bank debits. Its aim is to reduce repetitive handling while preserving source evidence and a record of human decisions; time savings have not yet been measured.

Current scope: one bookkeeper account; web and Telegram receipt intake; review, reporting, Excel export and monthly bank reconciliation.

Current limits: Other users cannot create their own accounts. Ledgerly does not make payments or post entries to accounting software; a person checks uncertain results.

2. Key objectives

- **Faster processing.** Extract receipt details and reuse known vendor categories.
- **Consistent records.** Apply a fixed category list and the same review rules to every receipt.
- **Human control.** Send uncertain results for review and retain corrections with an audit trail.
- **Useful outputs.** Show spending, compare bank debits and export records to Excel.
- **Practical deployment.** Run the controlled pilot on Lightsail and track processing costs.

3. Requirements

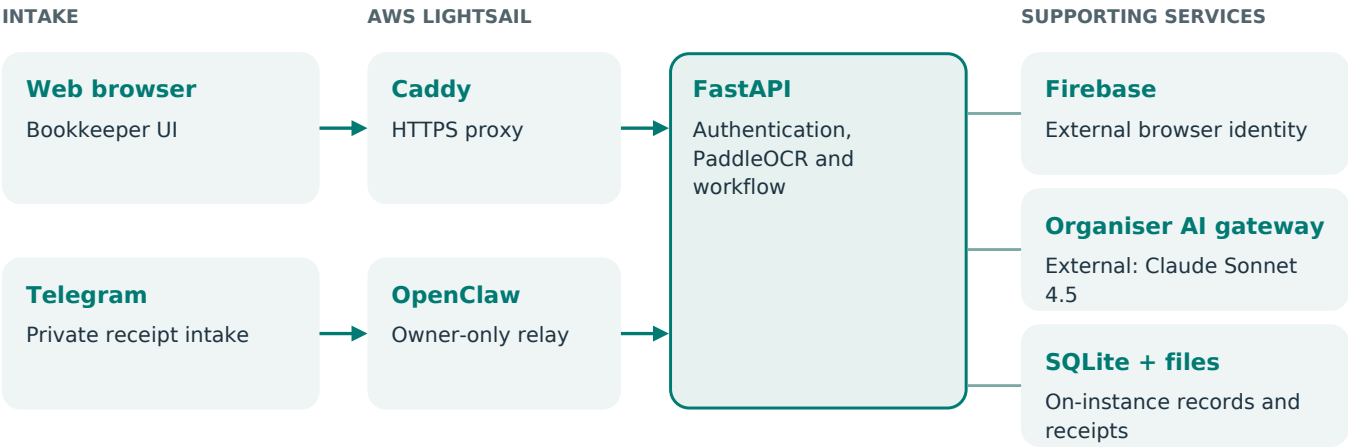
Functional

Function	What Ledgerly does
Receipt intake	Accept JPEG, PNG and bounded PDF files from the web app; accept Telegram uploads through the same backend.
Extraction and routing	Read receipt text locally, use a known-vendor mapping where possible, then suggest a category and route uncertain or duplicate-like records for review.
Review and reporting	Show source evidence; let a person approve, reject or amend with a reason; provide history, dashboard totals and Excel exports.
Monthly close	Preview PDF or import normalized CSV bank statements, retain their sources, suggest matches and display exceptions for investigation.
AI assistance	Provide scoped explanations without approving records, altering payment states or posting transactions.

Non-functional requirements in the MVP

Access and privacy Allowed Firebase UID or trusted Telegram key; unauthenticated financial requests receive 401.	Bounded processing Upload, PDF, OCR and model limits; only bounded text or context reaches the gateway.
Integrity and recovery Versioned/audited SQLite writes; database plus uploads backed up, with an operator-verified off-instance copy.	Capacity limit One bookkeeper workspace and API worker; uptime and concurrent load have not been benchmarked.

4. System architecture



Arrows: intake requests | Plain lines: supporting services
Figure 1- AWS Lightsail deployment and external supporting services.

Caddy and FastAPI run in containers on AWS Lightsail; OpenClaw is a separate host service. Browser and Telegram intake reach FastAPI, which uses local PaddleOCR for images and scanned PDF pages by default. Embedded PDF text bypasses OCR; Tesseract is a configurable alternative. SQLite and receipt files persist on the application volume. Firebase and the organiser gateway are external; only bounded text or context reaches the gateway.

Claude Sonnet 4.5 is the configured default, not a live-model verification. The diagram reflects repository settings; reported health and access checks do not verify all host settings.

5. Component details

Component	Role in the system
React and TypeScript UI	Six workspace views: Main dashboard, Monthly close, Upload receipt, Pending reviews, Receipt history and Deleted receipts.
FastAPI and PaddleOCR	Validate uploads; use PaddleOCR for images and scanned PDF pages by default, or embedded PDF text where available. Tesseract is configurable.
Classification	Use a saved vendor mapping first. For an unmatched vendor, call the text model; the confidence gate routes uncertainty to review.
SQLite and evidence	Store extracted receipts, classifications, reviews, bank transactions and retained statement sources in SQLite; keep original receipt files in the application volume.
Caddy and OpenClaw	Caddy handles HTTPS. OpenClaw is a separate host service that relays the approved Telegram user to the API.

AI responsibilities

Document Intelligence

Receipt extraction; statement extraction with consent

Extract only

Classification and Compliance

Category suggestions and optional review guidance

Recommend only

Finance Copilot

Month-scoped explanations from stored evidence

Read only

Figure 2- AI responsibilities and authority. Validation, duplicate checks, vendor lookup and bank matching remain application rules; models cannot record a human decision.

6. Data design

Related tables	Stored information
receipts; line_items; classifications; vendor_category_mappings	Original receipt reference and OCR/extraction fields; ordered items; first category decision; known-vendor rules.
receipt_reviews; review_audit; receipt_amendments; amendment_audit; lifecycle_events; receipt_reprocessing	Human decisions, corrections, delete/restore/void history and re-extraction drafts. Earlier evidence remains traceable.
bank_statements; bank_transactions; receipt_payment_events; monthly_close_reviews	Retained PDF/CSV statement bytes, imported debits, human payment follow-up and month-review evidence fingerprint.
workspace_settings; fx_rate_snapshots	Default reporting currency and dated reference rates for labelled converted dashboard figures.

The application migrations define SQLite schema version 9. Relationships are keyed by receipt ID or statement ID. Receipt corrections are saved as new versions; matching is calculated from accepted receipts and bank debits rather than stored as a permanent payment fact. Bank debit amounts are stored in cents.

7. Interface design

User view	What it supports
Upload and Pending reviews	Add a receipt, see extraction, compare the original and record an approval or rejection.
Receipt history and Deleted receipts	Search and filter, inspect audit history, export selected records and restore eligible deletions.
Main dashboard	Review spending and receipt counts by date and currency; open the underlying accepted receipts.
Monthly close	Preview retained statements, check suggested matches and exceptions, record a month review and export Excel.

API groups: receipt upload/history/export; review and amendments; statement preview/import/reconciliation; dashboard; and scoped **/ai** explanations. Private routes use Firebase bearer authentication in the browser or an API key for the trusted relay. **/health** reports service liveness only.

Data movement and decision boundary

A validated receipt enters the application volume under a generated name. Embedded PDF text or PaddleOCR text feeds structured extraction; only bounded text goes to the model gateway. The application checks the result, looks up the vendor, evaluates confidence and possible duplicates, then stores the original extraction and routing outcome in SQLite. Dashboards and exports read the accepted effective record, including a later human amendment.

8. Detailed design

Receipt processing and review workflow

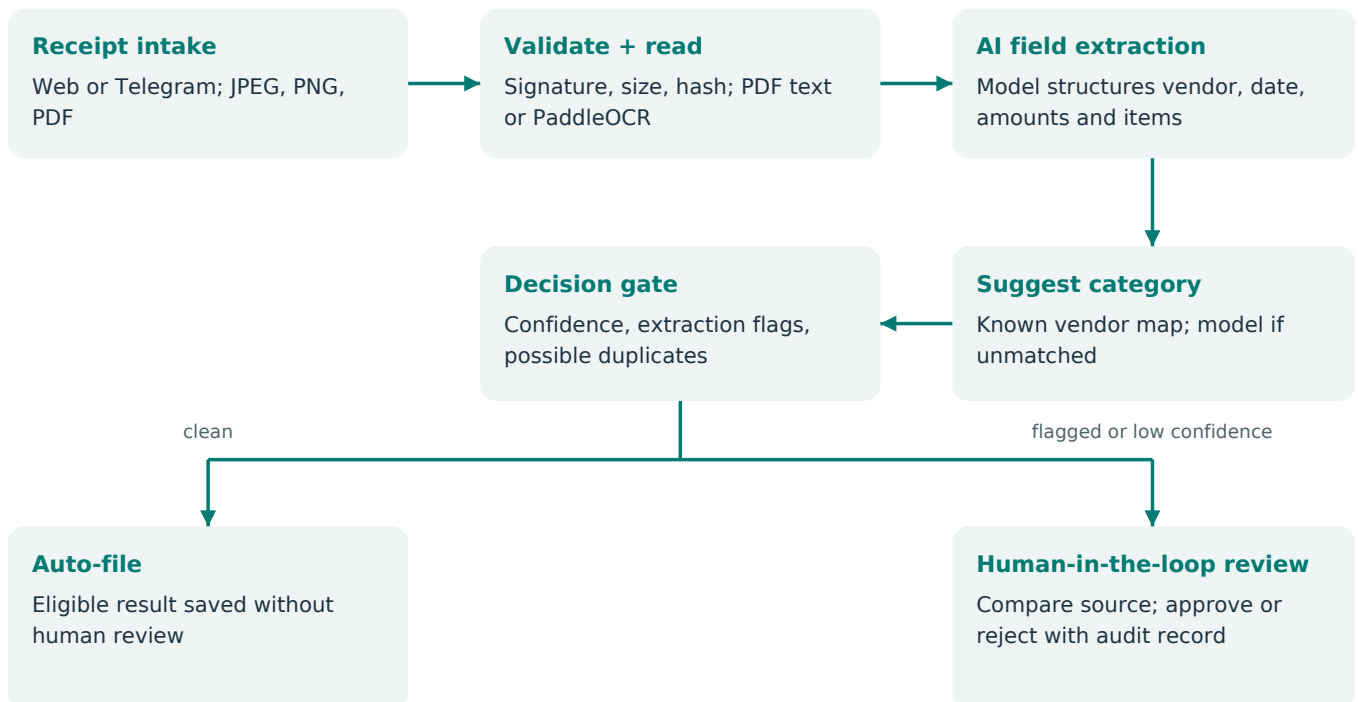


Figure 3- Receipt processing and review. PaddleOCR reads images or scanned PDF pages; native PDF text needs no OCR. AI extracts structured fields and the application validates them. Uncertain results enter human-in-the-loop review; eligible clean results auto-file.

- **Routing controls.** An identical retained file is rejected. A probable duplicate transaction, extraction warning or low-confidence classification enters Pending reviews, even if a known-vendor mapping supplies its category.
- **Human decision.** A reviewer compares original evidence, corrects values if needed and saves a reason. Approvals, rejections and later amendments leave an audit record; re-extraction produces a draft until a person adopts it.
- **Failure behaviour.** An unavailable classifier routes a receipt to review. An extraction failure may leave saved OCR for a later draft; some early OCR/PDF errors remove the source and require a fresh upload.

Monthly close and bank-statement workflow

PDF / PREVIEW AND HUMAN CONFIRMATION

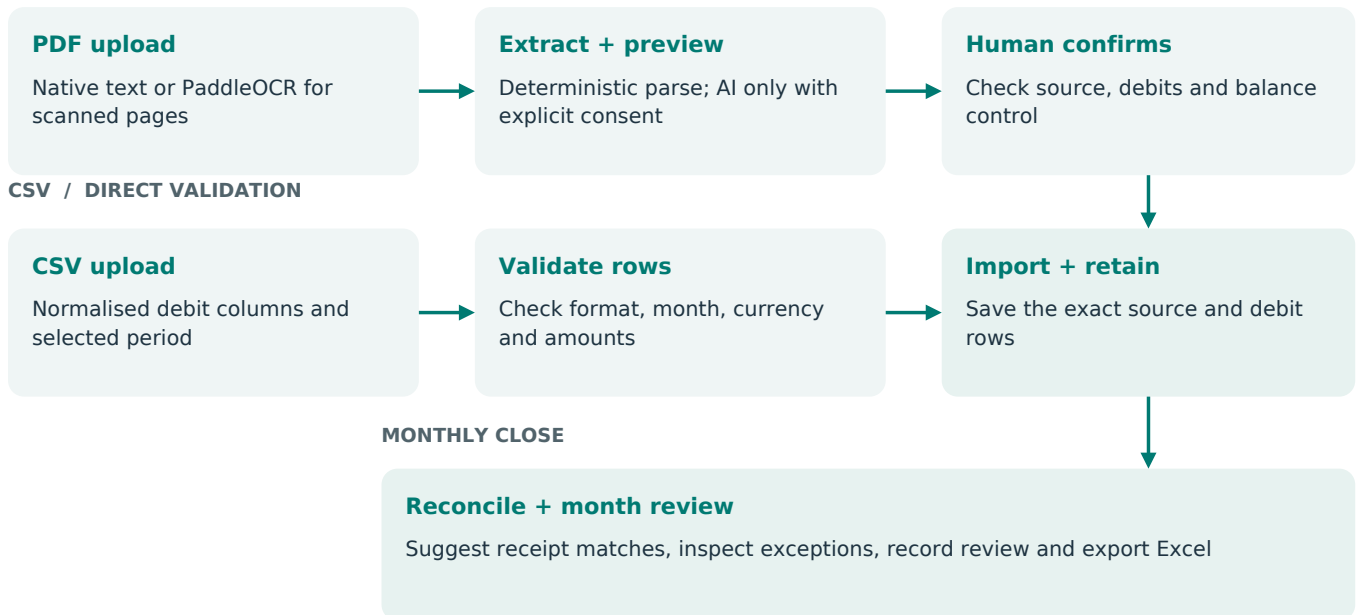


Figure 4- Monthly close workflow. PDF preview needs human confirmation; normalized CSV imports directly after validation. Both paths retain the source and feed deterministic reconciliation.

- **PDF evidence.** Use native text or PaddleOCR on scanned pages, then parse locally. Unsupported layouts can use AI only with explicit consent. A signed preview binds the file and extracted rows; a failed balance control blocks confirmation.
- **CSV route.** Validate normalized debit columns and import directly. Both import paths retain the original statement bytes and debit rows for source inspection.
- **Bank comparison.** Match accepted receipts by currency, exact amount and nearby date. Ambiguous or duplicate debits stay exceptions; a suggested match is not proof of payment.
- **Monthly review.** A person reviews exceptions and source evidence, records the month review, then exports Excel. Changes to receipts or statements make the saved review outdated.

Security and trust limits

The production Compose configuration binds the API to 127.0.0.1 while Caddy publishes HTTPS. The browser needs a Firebase ID token for the configured UID; the Telegram relay uses a separate API key. Authentication precedes file processing, and file signatures, size and PDF pages are bounded. Model responses are schema-validated and cannot approve, pay, post or change the stored month review. Reviewer names entered in the UI are self-reported. These controls describe code and configuration; the live host needs separate checks.

9. Testing plan and evaluation

Check	Observed evidence and current limit
Authentication and uploads	Operator reported HTTP 401 for unauthenticated receipts on 24 September 2026. Repository tests cover identity and invalid uploads; authenticated production intake was not rechecked for this revision.
Routing and review	Tests cover vendor mappings, uncertain/duplicate routing and stale or repeated decisions. These code-level cases do not measure real-world classification accuracy.
Reports and statements	Tests cover signed previews, balance checks, retained statement sources, exceptions, dashboard and exports. No new authenticated production statement/export run was made for this revision.
Deployment checks	Operator reported a healthy API, nine retained receipts and a checksum-verified off-instance backup on 24 September 2026. The operator confirms that web and Telegram receipt intake work; no fresh restart recovery drill was performed for this revision.

Evaluation status

GitHub Actions CI passed for main commit 9c04045 on 25 September 2026: backend tests, frontend checks, documentation checks and the Python dependency audit. This PDF revision did not rerun those checks locally or inspect the authenticated live UI. Accuracy, time saved and production capacity remain unmeasured on a representative dataset; nine retained receipts do not establish those outcomes.

Evaluation method

Freeze a labelled set of permitted receipts and vendor mappings before testing. Report sample size, field and category accuracy against labels, auto-file and review shares over all submitted receipts, end-to-end time including human review, errors and model calls per receipt. Compare a matched manual task and reconcile sampled Excel totals with effective accepted records. Record infrastructure and gateway cost for the same period; report failures separately.

Proposal targets, still unverified: at least 90% field extraction and category accuracy, at least 70% auto-file, at most 30% human review, at least 95% end-to-end processing success, and no more than US\$100 in shared infrastructure/API spending. Report denominators and costs alongside measured results.

10. Maintenance plan

- **Change and test.** Fix issues in a branch, run backend, frontend and documentation checks, then merge through the protected main branch.
- **Back up before updates.** Save the SQLite database and retained receipt files together; keep a verified copy off the Lightsail instance.
- **Deploy and check.** Use both production Compose files. After updates, confirm sign-in, a receipt flow, monthly close, export and retained evidence.
- **Respond to faults.** Monitor health, logs, disk space and costs. If a database migration changes the schema, restore the matching database and file backup with a compatible application image.

Deployment verification and recovery

Build and start with `compose.yaml` plus `compose.production.yaml`. Check the configured public HTTPS entry point, an unauthenticated 401 on financial routes, owner sign-in, a permitted receipt upload, retained source preview, a month review and an Excel export. A consistent SQLite copy and receipt uploads must be backed up together and copied off-instance. If the schema changed, restoring only an old image against a newer database is unsafe; recover the matching data and image together.

11. Conclusion and future work

Ledgerly provides one controlled workspace for receipt intake, review and monthly bank comparison. The operator reports working web and Telegram receipt intake on the Lightsail pilot. Its effect on bookkeeping time and classification accuracy still needs a labelled evaluation.

Proposed feature	How it would improve the current MVP
Resilient processing	Queue long OCR and model tasks with progress, controlled retries and idempotent writes so an interrupted upload can resume safely.
Safe public demo	Add a separate resettable workspace with synthetic receipts and statements so visitors can try Ledgerly without accessing the bookkeeper's records.
Team access	Introduce distinct reviewer accounts, role-based permissions and isolated workspaces, with audit actors bound to authenticated identities.
Accounting integration	Add review-gated posting to an accounting platform after category and account mapping validation; retain evidence and correction history.
Operational resilience	Automate protected backups and recovery checks, add alerts for failures and move the pilot to an owned domain before wider use.

Design checked against Ledgerly GitHub main at commit 9c04045, including UI navigation, database migrations, Compose configuration and test files. GitHub Actions passed all four jobs for that commit on 25 September 2026. The operator reported a healthy API, HTTP 401 for unauthenticated receipts, nine retained receipts and a checksum-verified off-instance backup on 24 September 2026. Those host observations are point-in-time reports; the live database, authenticated UI and complete host configuration were not directly inspected for this revision.

Source and evidence map

Implementation: app/main.py, app/database.py, app/review.py, app/statements.py, app/agents/ and frontend/src/.
Deployment: compose.yaml, compose.production.yaml and deploy/Caddyfile. **Verification:** tests/, frontend/tests/, scripts/check_docs.py, docs/architecture.md and docs/security.md. These paths support the implementation claims; measured outcomes require separate evidence.

Repository revision: [main commit 9c04045](#). **CI record:** [25 September 2026 workflow run](#). The original hackathon proposal supplies target metrics; the current code and documentation supply implementation facts.